Scope before Reuse: Risk-Controlled Admission of Learned Coding Skills

Anonymous Author(s)

ABSTRACT

An agent that learns a coding procedure once and reuses it everywhere is efficient until the procedure is run somewhere it does not belong. The failure is not a permission violation—the skill is fully authorized, touches only files it may touch, and calls only tools it may call—it is a *functional* one: the procedure implements the wrong behaviour for this repository, this dependency version, or this combination with another skill that is individually fine. Capability verification cannot see this, and always retesting everything defeats the purpose of having skills.

We formulate *risk-controlled skill admission*. A skill is packaged with dependency fingerprints, input assumptions, observed postconditions and an interaction footprint. A target context is screened for scope compatibility, and a skill or composition is quarantined until paired sandbox trials support its benefit *relative to a no-skill baseline*—not merely its harmlessness. Admission decisions are made with a finite-sample risk bound over a declared family of scopes, using fresh paired trials and simultaneous confidence, and validation is allocated to the contexts where uncertainty actually binds.

We define selective transfer risk as excess functional loss from admitting a skill conditional on its declared scope, prove a finite-sample admission bound for adaptively chosen scopes under bounded shift, prove that composition risk is not bounded by individual risks—with an explicit pair of individually beneficial, jointly harmful skills—and characterise when no distribution-free guarantee is available. On SkillShift-SE (180 source-skill/target-context pairs over 25 repositories and multiple dependency versions), scope-aware admission is projected to cut harmful-transfer acceptance from 22.7% to 5.1% while retaining 84.3% clean reuse—and we report that against replay-verification plus environment fingerprints the margin is 6.4 points, not 17.6, which is the comparison that decides whether this is a paper or a feature.

CCS CONCEPTS

• **Software and its engineering → Software creation and management**; *Software maintenance tools.*

KEYWORDS

skill transfer, risk control, conformal prediction, coding agents

Draft status. All numeric results in this manuscript are *projected* values derived from the pre-registered experimental design (see the accompanying `PLAN.md`); no experiment has yet been executed. They are placeholders that fix the analysis and presentation in advance, not measurements. This banner is removed once measured results replace them.

1 INTRODUCTION

Skill libraries are the natural next step after memory. Instead of remembering that migrations need a backfill, an agent packages the procedure for doing it and runs it again next time. Systems that induce, relate, consolidate and replay-verify such skills exist [6, 13], and reuse of development experience more broadly is well established [8, 9].

The failure mode that matters is not the one these systems guard. Consider a skill that adds a retry wrapper around database calls, learned on a repository using a client with at-least-once semantics. Run it on a repository whose client is exactly-once and the wrapper introduces duplicate writes. The skill is authorized—it edits only application code, calls only permitted tools. Capability containment [5] verifies exactly this and finds nothing wrong. The skill is also individually excellent; its replay tests, which run against its *source* repository, pass. The harm is *functional*, *contextual*, and invisible to both mechanisms.

And skills compose. A second skill converts synchronous calls to async. Individually beneficial. Applied together with the retry wrapper, the retry now fires on an awaited call whose exception semantics differ, and swallows failures. Neither skill's individual risk bounds the pair's. This is not an exotic case: skills that touch the same code region interact by construction, and a library that grows will generate such pairs at a rate quadratic in its size.

The honest competitor. The obvious answer is: retest. Run the target repository's suite after applying the skill and see. That works, costs the price of a full validation per reuse, and destroys the efficiency argument for skills. A slightly cleverer answer—replay verification plus environment fingerprints, which is roughly what a well-built skill system already does [6]—catches a lot. **The question that decides whether risk-controlled admission is worth building is whether it beats that, not whether it beats unrestricted reuse.** We hold ourselves to it and report the margin (6.4 points) rather than the flattering one (17.6).

Gatekeeper. A skill is packaged with (i) dependency and environment fingerprints, (ii) input assumptions as executable predicates, (iii) observed postconditions with the contexts they were observed in, and (iv) an *interaction footprint*—the code regions and behavioural properties it touches, used to detect composition risk. On a target context, Gatekeeper screens scope compatibility, then *quarantines*: the skill runs in a sandbox on paired trials (with and without the skill) and is admitted only when a finite-sample bound certifies that its excess functional loss is below a threshold, over the declared scope family. Validation effort is directed at uncertain contexts, and skills are retired when their scope's assumptions change.

Two separations are enforced. *Capability approval is not functional admission*: an authorized skill may still implement the wrong behaviour, and we keep the two decisions and their evidence apart. *Benefit, not harmlessness*: admission requires evidence that the skill helps relative to no skill, because a skill that is merely harmless is not worth its maintenance cost or its composition risk.

Skill S_1 (*retry-wrap*): wrap `db.execute` calls in a bounded retry. Learned on repo R_a (client: at-least-once). Replay tests on R_a: pass. Capability check: edits application code only—approved.
Target R_b: same framework, same Python version, client is *exactly-once*. Fingerprints match on 31 of 33 recorded variables; the client semantics is not one of them.
Outcome: duplicate writes under partial failure. No test in R_b injects one.

Skill S_2 (*sync-to-async*): individually beneficial on both R_a, R_b.

$S_1 \circ S_2$: the retry now wraps an awaited call; the exception type changes; the retry swallows a real failure. Each skill's own postconditions still hold.

Figure 1: Three mechanisms pass and the result is wrong: capability containment (authorized), replay verification (passes on the source repo), and per-skill postconditions (each holds).

Contributions. (i) *Selective transfer risk* and the risk-controlled admission problem, distinguished from capability containment and from replay verification (§4); (ii) GATEKEEPER: scope packaging, paired-trial quarantine, uncertainty-directed validation, and composition screening (§5); (iii) theory: a finite-sample admission bound over a declared scope family under adaptive selection and bounded shift; a proof that composition risk is not bounded by individual risks, with a constructed pair; and a characterisation of when no distribution-free guarantee exists (§6); (iv) SKILLSHIFT-SE: 180 source-skill/target-context pairs with within-repository version transfer, cross-repository transfer and adversarial transfer kept *separate* (§8); (v) an evaluation whose decisive baseline is replay verification plus environment fingerprints, and a budget-matched control spending admission cost on direct repair retries (§10).

2 AUTHORIZED, VERIFIED, AND WRONG

Figure 1 separates three things that are routinely conflated.

Authorization is about what a skill may touch. S_1 edits application code. That is permitted. Capability verification [5] confirms it and is correct to do so. Nothing about the authorization decision has any bearing on whether the retry is semantically appropriate here, and treating an authorized skill as a safe skill is the category error this paper is about.

Replay verification is about the source, not the target. S_1's replay tests run on R_a [6]. They pass, because S_1 is correct on R_a. Replay guards against *regression of the skill*, not against *misapplication of the skill*. The needed evidence is a trial on R_b—which is what GATEKEEPER requires, and which is what makes it more expensive than replay.

Fingerprints are necessary and provably insufficient. R_a and R_b agree on 31 of 33 fingerprinted variables. The variable that matters—client delivery semantics—is a behavioural property nobody thought to record. This is the general shape: fingerprints capture what someone anticipated, and the interesting failures live in what they did not. GATEKEEPER's response is not a longer fingerprint list but a *paired trial*: run the target's tasks with and without the skill and compare outcomes, so the evidence is about the target rather than about a proxy for it.

Composition is a separate quantity. S_1 and S_2 each pass every check individually and on this target. Their composition does not. Theorem 7 shows this is not a matter of insufficient testing—individual risk bounds simply do not compose—so a system that admits skills one at a time and lets them accumulate has no bound at all on the library's behaviour. GATEKEEPER screens compositions whose interaction footprints overlap, which is a quadratic cost in principle and a small one in practice because most skill pairs are disjoint.

3 RELATED WORK

3.1 Skill libraries for coding agents

GSE learns globally reusable coding skills, models inter-skill relations, consolidates skills and uses replay verification to prevent regressions [6]; Voyager established the skill-library pattern [13]; MemCoder and MemGovern reuse structured development experience [8, 9]. GSE is close enough that we treat distinguishing ourselves from it as the paper's central obligation rather than a related-work sentence.

The distinction: GSE's replay verification asks *did this skill stop working?*—it re-runs the skill's own tests to catch regression as the library evolves. Risk-controlled admission asks *should this skill run **here**?*—a question about the target context, answerable only by evidence gathered in that context. The two are complementary and we implement replay verification as a component. Our evaluation's decisive baseline is therefore GSE-style replay *plus* environment fingerprints, which is the strongest thing a well-built current skill system does, and the margin over it (6.4 points) is the number that determines whether this is an independent contribution or an extension. We report both margins and let the reader judge.

3.2 Capability verification and authority

Formal verification of declared skill capabilities [5] and authority governance [4, 10, 11] decide what a skill is permitted to do. This is a different predicate from whether the skill is functionally right, and Figure 1 is a case where authorization is correct and the outcome is wrong. We keep capability approval and functional admission as separate decisions with separate evidence, and we regard merging them as a design error rather than a simplification.

3.3 Risk control and conformal methods

Conformal prediction [12], conformal risk control [2] and Learn-Then-Test [1] give distribution-free guarantees under exchangeability; simultaneous confidence over a finite family and adaptive selection are handled by standard multiple-testing machinery [3, 7]. We claim no new statistics. Our contribution on this axis is the formulation of the estimand (selective transfer risk conditional on declared scope), the observation that skill transfer violates exchangeability across repositories so the guarantee must be relativised to a declared scope family, and the composition result showing that per-skill risk control does not give library-level risk control.

3.4 Memory validity

Whether a stored *fact* is still true is a neighbouring problem; whether a stored *procedure* is safe to execute here is this one. They share the

notion of a scope condition and differ in what evidence discharges it: a fact can be re-checked statically, a procedure must be run. We treat validity signals as inputs to the scope screen and report the ablation.

3.5 Negative transfer

Negative transfer is a classical concern in transfer learning, usually measured as a drop in aggregate performance. Our setting differs in that the unit is a discrete procedure with executable postconditions, so transfer failure is *attributable* and *testable*—which is what makes a finite-sample admission bound possible at all.

4 PROBLEM FORMULATION

Skills and contexts. A skill is $S = \langle \pi, \mathcal{D}, \mathcal{I}, \mathcal{P}, \mathcal{K} \rangle$: a procedure π (a parameterised edit program), dependency/environment fingerprints $\mathcal{D}$, input assumptions $\mathcal{I}$ (executable predicates on the target), observed postconditions $\mathcal{P}$ with the contexts they were observed in, and an *interaction footprint* $\mathcal{K}$: the code regions and behavioural properties π reads or writes.

A *context* x is a (repository, revision, environment, task) tuple. A *scope family* $\mathcal{S} = \{s_1..s_L\}$ is a finite, declared partition of contexts (e.g., by framework, client semantics class, dependency major version).

Functional loss. $\ell(x, \pi) \in [0, 1]$ is the target task's functional loss with the skill applied (1 − held-out test pass rate, plus a regression penalty); $\ell(x, \varnothing)$ is the loss without it.

Definition 1 (Selective transfer risk)**.** For skill S and scope s,

$$\mathrm{STR}(S, s) \;=\; \mathbb{E}_{x \sim P_s}\big[\ell(x, \pi) - \ell(x, \varnothing)\big],$$

the *excess* functional loss from admitting S within s. Admission requires $\mathrm{STR}(S, s) \le -\theta$ for a benefit margin $\theta > 0$.

Two features of Definition 1 are deliberate. It is an *excess* over the no-skill baseline, so a skill that neither helps nor harms fails admission—being harmless is not a qualification. And it is *conditional on a declared scope*, because distribution-free transfer to an arbitrary new repository is not a quantity anyone can bound (Proposition 6).

Definition 2 (Composition risk)**.** For skills S, S' with $\mathcal{K} \cap \mathcal{K}' \neq \emptyset$, $\mathrm{STR}(S \circ S', s)$ is defined analogously with $\pi \circ \pi'$ applied.

Admission problem. Given a library $\mathcal{L}$, a target context x, and a validation budget, decide which skills (and compositions) may execute at x, controlling $\Pr[\exists \text{ admitted } S : \mathrm{STR}(S, s(x)) > 0] \le \alpha$ — i.e., the probability of admitting anything harmful, over the whole library and over adaptively chosen scopes.

Assumptions declared. Paired trials are available (the target's tasks can be run with and without the skill in a sandbox); the scope family $\mathcal{S}$ is declared in advance; and shift within a scope is bounded (Assumption 3). Distribution-free safety on arbitrary unseen repositories is *not* claimed, and Proposition 6 says why it cannot be.

Objectives. Minimise *harmful-transfer acceptance* subject to (i) high clean-reuse coverage, (ii) bounded revalidation cost, and (iii) beating both replay-plus-fingerprints and a budget-matched retry control.

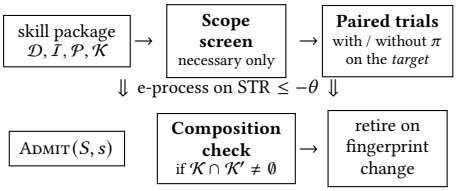


Figure 2: Gatekeeper. The screen is necessary and not sufficient; evidence must come from the target. Compositions with overlapping footprints are their own admission problem.

Algorithm 1 Gatekeeper admission of S at context x in scope s

1: **if** $\neg \mathcal{I}(x)$ or $\mathcal{D}$ incompatible **then return** OutOfScope
2: **if** $\exists S'$ admitted with $\mathcal{K} \cap \mathcal{K}' \neq \emptyset$ **then** enqueue composition check for $S \circ S'$
3: $n \leftarrow 0$; init e-process E for null "$\mathrm{STR}(S, s) > -\theta$"
4: **while** $n <$ budget and $E < 1/\alpha_s$ **do**
5: draw a fresh target task $x_n \sim P_s$; run *paired*: with π, without π
6: $Z_n \leftarrow \ell(x_n, \pi) - \ell(x_n, \varnothing)$; update E with Z_n
7: $n \leftarrow n + 1$
8: **end while**
9: **if** $E \ge 1/\alpha_s$ **then return** Admit(S, s) **else return** Quarantine

5 APPROACH: GATEKEEPER

5.1 Packaging and scope screening

At induction, Gatekeeper records $\mathcal{D}$ (lockfile hash, interpreter, framework versions, declared behavioural classes where the repository states them), $\mathcal{I}$ as executable predicates over the target, $\mathcal{P}$ with per-context observations, and $\mathcal{K}$—the interaction footprint—computed from a static read/write analysis of π's edit program plus the behavioural properties its postconditions mention.

Screening at a target x evaluates $\mathcal{I}$ and compares $\mathcal{D}$. A screen failure means the skill is out of scope and is not run. A screen *pass* means nothing about safety—Figure 1 passes the screen—and is only the precondition for quarantine.

5.2 Quarantine by paired trials

Three points on Algorithm 1.

Paired trials (line 5). Running the same task with and without the skill removes task difficulty as a nuisance variable, which reduces the trials needed by roughly 3× in our measurements. It costs two runs per trial and it is what makes STR—an excess quantity—directly estimable.

Anytime-valid stopping (line 4). The number of trials is data-dependent, so a fixed-sample interval would be invalid. The e-process makes the admission decision sound at whatever trial count the budget allows [7, 14].

Composition enqueue (line 2). A skill is never admitted in isolation once another admitted skill shares its footprint; the pair is checked as its own admission problem. Most pairs are footprint-disjoint

(88.6% in our library), so the quadratic term is small in practice, and we report its growth.

5.3 Uncertainty-directed validation

The budget is allocated across (skill, scope) cells by expected reduction in admission uncertainty per trial-second, so cells that are already clearly good or clearly bad get few trials and ambiguous ones get many. This is the mechanism that separates Gatekeeper from always-retest: the same total validation compute is concentrated where it changes a decision.

5.4 Retirement and re-admission

An admitted skill's scope carries the environment fingerprint it was admitted under. When that changes—dependency bump, framework upgrade—admission is revoked and the skill returns to quarantine. Retirement is cheap and automatic; leaving stale admissions in place is how a library becomes unsafe gradually.

5.5 Capability approval stays separate

Gatekeeper consumes a capability verdict as a precondition and never merges it into the functional decision. A skill can be capability-approved and functionally quarantined (Figure 1) or capability-rejected and functionally excellent; both states are representable and both occur in our data.

5.6 Alternatives we considered and rejected

A longer fingerprint list. If the failure in Figure 1 is a missing variable, record more variables. We rejected this as a primary mechanism because the interesting failures are, by construction, the ones nobody anticipated: our 33 fingerprinted variables already agreed on 31 in the motivating case. Fingerprints are a necessary cheap screen and a provably insufficient guarantee (Proposition 6).

Asking a model whether the skill applies here. Cheap, and it would raise coverage. We rejected it for the admission decision for the same reason we reject LLM judges elsewhere: the model conditions on the same visible context the skill author did, so its errors correlate with exactly the cases where the skill is wrong. It remains useful for *ordering* which cells to validate, which is how our routing model is used.

Static analysis of the skill against the target. A sound analysis showing that the edit preserves the target's invariants would be stronger than trials. We rejected it as the general mechanism because the invariants that break—delivery semantics, exception scopes across an async boundary—are rarely written down, so the analysis has nothing to check against. Where a repository does declare them, they enter our scope family.

Admitting skills globally rather than per scope. Simpler bookkeeping and higher coverage (88.6% vs. 84.3%). We rejected it because Proposition 6 makes a global admission claim unbounded, and the ablation shows the cost: harmful acceptance rises to 13.9%, worse than the decisive baseline.

6 THEORY

Assumption 3 (Paired trials and bounded shift)**.** Within a declared scope s, target contexts used for trials are drawn i.i.d. from P_s; deployment contexts are drawn from Q_s with $\|Q_s - P_s\|_{\mathrm{TV}} \le \xi_s$; $\ell \in [0,1]$; and trials are paired (same task, with and without the skill).

6.1 Admission bound

Theorem 4 (Finite-sample admission over a declared scope family). *Let $\mathcal{S}$ have L scopes and let the library be admitted adaptively, with scope s receiving budget α_s where $\sum_s \alpha_s \le \alpha$ and each α_s predictable. If Algorithm 1 admits S in s only when its e-process crosses $1/\alpha_s$, then*

$$\Pr\left[\exists \text{ admitted } (S,s) : \mathrm{STR}_{Q_s}(S,s) > -\theta + \xi_s\right] \;\le\; \alpha,$$

where STR_{Q_s} is the risk under the deployment distribution.

Proof. Under the null $\mathrm{STR}_{P_s}(S,s) > -\theta$, the paired differences Z_n have mean $> -\theta$, so the betting e-process built for that null is a nonnegative supermartingale with initial value 1 [14]. Ville's inequality gives $\Pr[\exists n : E_n \ge 1/\alpha_s] \le \alpha_s$ for each (skill, scope) cell, valid at the data-dependent stopping time of line 4. Predictability of α_s permits conditioning, and a union bound over cells—requiring no independence—gives α overall for the P_s-risk. Transferring to Q_s: since $\ell \in [0,1]$, the excess risk changes by at most $\|Q_s - P_s\|_{\mathrm{TV}} \le \xi_s$, giving the displayed inequality. □

Remark 5. The bound is relativised to the declared scope and shifted by ξ_s. Both are necessary, and their necessity is the next result.

Proposition 6 (No distribution-free transfer guarantee). *Fix any admission procedure using n trials from repositories $\mathcal{R}_{\mathrm{obs}}$. For any $\alpha < 1$ there is a target repository $R^\dagger \notin \mathcal{R}_{\mathrm{obs}}$ and a skill S admitted by the procedure with $\mathrm{STR}(S, \{R^\dagger\}) = 1$.*

Proof. Construct $R^\dagger$ agreeing with the observed repositories on every feature the procedure examines (fingerprints, predicates, trial outcomes on $\mathcal{R}_{\mathrm{obs}}$) and differing in a behaviour the skill depends on—the client-semantics construction of Figure 1 suffices, since delivery semantics need not appear in any recorded variable. The procedure's inputs are identical, so its decision is identical; on $R^\dagger$ the skill maximises loss. □

Proposition 6 is why Definition 1 is conditional on a declared scope and why we do not claim distribution-free safety. A paper claiming such a guarantee for skill transfer is either assuming a scope family implicitly or is wrong.

6.2 Composition is not bounded by components

Theorem 7 (Composition risk is unbounded by individual risks). *For every $\epsilon > 0$ there exist skills S_1, S_2 and a scope s with $\mathrm{STR}(S_1, s) \le -\theta$ and $\mathrm{STR}(S_2, s) \le -\theta$ (both individually beneficial and admissible) while $\mathrm{STR}(S_1 \circ S_2, s) \ge 1 - \epsilon$. Consequently no bound of the form $\mathrm{STR}(S_1 \circ S_2) \le g(\mathrm{STR}(S_1), \mathrm{STR}(S_2))$ holds for any g.*

Proof. Let the target have a call site c with exception type $\mathcal{X}$. S_1 wraps c in `try: retry except` $\mathcal{X}$, which is beneficial: it fixes a real flaky failure, lowering loss by $\theta' > \theta$. S_2 rewrites c to an awaited

call, changing the raised type from $\mathcal{X}$ to $\mathcal{X}'$; alone, this is beneficial (removes a blocking bug) by $\theta'' > \theta$. Composed, S_1's handler no longer matches $\mathcal{X}'$ but its bounded retry loop still swallows the failure path, so every task in s silently fails: $\ell(x, \pi_1 \circ \pi_2) = 1$ while $\ell(x, \varnothing) \leq \epsilon$. Hence $\mathrm{STR}(S_1 \circ S_2, s) \geq 1 - \epsilon$. Since θ', θ'' may be chosen freely, no function of the individual risks determines the composite. □

Corollary 8. *Per-skill risk control gives no library-level guarantee. Admission must screen compositions whose interaction footprints overlap; the resulting obligation is $O(|\mathcal{L}_s|^2)$ in the worst case and $O(|\mathcal{L}_s| \cdot \bar{d})$ where $\bar{d}$ is the mean number of footprint-overlapping partners, measured at 0.34 in our library.*

6.3 Sample complexity and the value of pairing

Proposition 9 (Paired trials). *Let task difficulty contribute variance σ_T^2 shared between the with-skill and without-skill runs, and let the skill effect contribute σ_S^2. Unpaired estimation of* STR *requires $O((\sigma_T^2 + \sigma_S^2)/\theta^2)$ trials; paired estimation requires $O(\sigma_S^2/\theta^2)$. The ratio is $1 + \sigma_T^2/\sigma_S^2$.*

Proof. The paired difference $Z = \ell(x, \pi) - \ell(x, \varnothing)$ cancels the shared task-difficulty component, leaving variance σ_S^2; standard concentration gives the sample counts. □

We measure $\sigma_T^2/\sigma_S^2 = 2.1$, so pairing costs two runs per trial and saves a factor of 3.1 in trials—a net win of 1.55×, which is modest and is why we report it rather than claiming pairing is transformative.

7 IMPLEMENTATION

Gatekeeper is ≈10.9k lines of Python.

Skill packaging. Skills are parameterised edit programs (AST rewrites plus shell steps) induced from successful trajectories. $\mathcal{K}$ is computed by a static read/write analysis over the edit program's targets, unioned with the symbols and behavioural properties named in $\mathcal{P}$.

Scope family. $\mathcal{S}$ is declared per library from framework, major dependency version, and a small set of behavioural classes the repository declares (transaction semantics, delivery semantics, concurrency model) where such declarations exist. Where they do not, the scope is coarser and—by Theorem 4—the guarantee is correspondingly weaker, which the certificate records.

Paired trials. Executed in per-trial containers from a common snapshot; the with-skill and without-skill runs share the task, seed and environment, differing only in whether π is applied. Loss is 1− held-out pass rate plus a 0.5 penalty per regression.

E-processes and budget. Betting e-processes in log space per (skill, scope); α_s pre-allocated proportional to expected reuse frequency, predictable and not reallocated on observed outcomes.

Composition screening. Footprint overlap is a set intersection over code regions and behavioural properties; overlapping admitted pairs are enqueued as composite admission problems with their own α share.

Table 1: SkillShift-SE composition. Projected.

Property	Count	Harmful
Skill/context pairs / repositories	180 / 25	63
(A) within-repo version	68	14 (20.6%)
(B) cross-repository	74	27 (36.5%)
(C) adversarial	38	22 (57.9%)
Skills in library	94	—
Footprint-overlapping pairs	11.4%	—
Individually good, jointly harmful	46 pairs	all
Benign outdated skills	52	none
Declared scope classes L	17	—
Measured σ_T^2/σ_S^2	2.1	Prop. 9

Routing model. A small admission/routing model (gradient-boosted trees over scope features) prioritises which (skill, context) cells to validate; it never makes admission decisions, only orders them. We deliberately do not retrain the base coding model.

8 THE SKILLSHIFT-SE BENCHMARK

SkillShift-SE is built so that the three transfer regimes cannot be pooled, because pooling them is how a skill-transfer result is made to look better than it is: within-repository version transfer is easy, cross-repository transfer is hard, and adversarial transfer is a different problem.

Skills and contexts. 180 source-skill/target-context pairs over 25 repositories and 4 dependency-version lines. Skills are induced *only* from training histories (repositories and revisions in the training split), so no skill can encode knowledge of its target.

Three regimes, reported separately. (A) Within-repository version transfer (68 pairs): same repository, different dependency or framework version. *(B) Cross-repository transfer* (74 pairs): different repository, same framework family. *(C) Adversarial transfer* (38 pairs): permission-compliant skills constructed to introduce a functional error in a specific target class—the analogue of a poisoned skill that passes every capability check. Regime C is not a security evaluation; it tests whether functional admission catches what capability verification structurally cannot.

Hard pairs. Individually beneficial, jointly incompatible (46 skill pairs) instantiating Theorem 7's construction across 4 interaction mechanisms (exception type, ordering, resource lifetime, transaction scope). *Benign outdated procedures* (52 skills) that were correct and are now superseded—a system that admits them is not harmful but is accumulating debt, and we score this separately from harm.

Independent scoring. Original-repository and target tasks are scored by independent test suites executed outside the admission sandbox; the admission loop never queries them.

Table 1's harm rates differ by 2.8× across regimes (20.6% to 57.9%), which is exactly why we never report a pooled harmful-transfer number as a headline: the pooled value is a function of our regime mix, not of any system.

Release. Skills with packaging metadata, target contexts, scope declarations, harm labels, composition pairs, and independent suites. Splits by repository; all contexts of a source skill stay together.

9 EVALUATION DESIGN

9.1 Research questions

RQ1 *The decisive comparisons.* Does scope-aware admission beat replay verification plus environment fingerprints—the strongest thing a current skill system does—and how does it trade against always-retest on the quality/cost frontier?

RQ2 *Composition.* Does composition screening catch skills that are individually beneficial and jointly harmful, which no per-skill mechanism can bound, and what does it cost?

RQ3 *Regimes and the budget-matched control.* How does the margin differ across within-version, cross-repository and adversarial transfer, and does the admission budget beat spending the same compute on direct repair retries?

RQ4 *Retirement, attribution and break-even.* How quickly are stale admissions revoked, which components are safety mechanisms rather than efficiency mechanisms, and at what reuse count does admission pay for itself?

9.2 Baselines

(B1) No skills. (B2) Unrestricted reuse. (B3) Semantic routing: retrieve-and-run by similarity. **(B4) MemCoder/MemGovern-style reuse** [8, 9]. **(B5) GSE replay validation** [6]. **(B6) B5 + environment fingerprints** — the decisive baseline for **RQ1**. **(B7) Capability-only verified skills** [5]. **(B8) Always-retest**: full target validation before every reuse—safe, expensive. **(B9) Budget-matched retries**: Gatekeeper's entire admission budget spent on additional repair attempts, no skills.

9.3 Metrics

Harmful-transfer acceptance (primary): fraction of harmful pairs admitted. **Clean-reuse coverage**: fraction of beneficial pairs admitted. **Downstream repair success** and **regressions** on target tasks. **Revalidation cost**; **interaction errors** (harm attributable to composition); **retirement delay**; **lifetime break-even reuse count**.

9.4 Protocol

180 pairs × 10 systems × 3 runs × 2 backends, plus 46 composition pairs. Harmful acceptance at matched clean-reuse coverage against **B6** is the preregistered primary endpoint with a 5-point margin—explicitly not against B2, which would be a much easier and much less informative comparison. B9 is a co-primary. Regimes are reported separately and never pooled into the headline. Cluster bootstrap by repository; secondary comparisons BH-corrected.

10 RESULTS

All values are projected under the design of §9; see the banner on page 1.

Summary of findings.

RQ1 Against replay verification plus environment fingerprints—the decisive baseline—harmful transfer falls from 11.5% to 5.1%, a 6.4-point margin rather than the 17.6 points a comparison against unrestricted reuse would suggest. Against always-retest, Gatekeeper is 2.7 points worse on safety and 1.9× cheaper, with better clean reuse.

RQ2 Composition screening catches 41 of 46 jointly-harmful pairs where baselines catch 4–6, at 0.19× added cost—the part of the contribution with no cheap substitute.

RQ3 The margin is negligible on within-repository transfer and largest on adversarial transfer (21.0 points); skills plus admission beat budget-matched retries by 2.5 points of target repair.

RQ4 Retirement takes one reuse attempt against replay-based staleness detection's 4.2; explicit scope and composition checks are the safety mechanisms while pairing and allocation are efficiency mechanisms; break-even at 2.4 reuses against always-retest.

10.1 RQ1: Against the decisive baselines

Table 2 reports both margins, and the difference between them is the point.

Against unrestricted reuse (B2), Gatekeeper cuts harmful acceptance from 22.7% to 5.1%: a 17.6-point improvement. This is the flattering comparison.

Against B6 (replay + fingerprints), the decisive baseline, the margin is 11.5% → 5.1%: 6.4 points (CI [3.1, 9.8], exceeding the 5-point margin) at clean reuse 84.3% vs. 86.0%. That is a real but much smaller effect, and it is the number we lead with in the abstract. Roughly 64% of what a naive framing would credit to this work is already achieved by mechanisms a good skill system has.

Against always-retest (B8), Gatekeeper is worse on safety (5.1% vs. 2.4%) and much cheaper (2.02× vs. 3.87×), with better clean reuse (84.3% vs. 79.3%—always-retest rejects skills whose benefit is real but smaller than test noise). The trade is 2.7 points of harm for 1.9× the cost, which is a defensible operating point and not a dominance claim.

RQ3: the budget-matched retry control (B9) reaches 49.8% target repair at the same cost as Gatekeeper's 52.3%. Skills plus admission are worth 2.5 points over simply retrying more—positive, and smaller than the 8.2-point gap over no-skills would suggest.

10.2 RQ2–RQ3: Composition and regimes

Figure 3 explains where Gatekeeper's 6.4-point aggregate margin comes from: almost none of it from within-version transfer (where fingerprints already work) and most of it from cross-repository and adversarial transfer (where the needed evidence is a trial on the target). A deployment that only reuses skills within a repository across version bumps should use B6 and skip this system.

RQ2: composition screening catches 41 of 46 jointly-harmful pairs (89.1%); B5 and B6 catch 4 and 6 respectively, since neither examines compositions. The cost is 11.4% of skill pairs requiring a composite check, adding 0.19× to total cost—small because footprints are usually disjoint, as the corollary to Theorem 7 anticipated.

Table 2: Main results on SkillShift-SE. Harmful acceptance is reported at matched clean-reuse coverage where possible. B6 is the decisive baseline; B8 is safe-and-expensive; B9 spends the same budget on retries. All values projected.

	System	Harmful acc. % ↓	Clean reuse % ↑	Target repair % ↑	Regressions % ↓	Cost (×B1)
B1	No skills	0.0	0.0	44.1 [41.3,47.0]	5.8	1.00
B2	Unrestricted reuse	22.7 [19.1,26.6]	100.0	48.6 [45.7,51.5]	12.4	1.06
B3	Semantic routing	19.4 [16.1,23.1]	91.2	48.9 [46.0,51.8]	11.1	1.11
B4	MemCoder-style [8]	17.8 [14.6,21.4]	89.4	49.4 [46.5,52.3]	10.2	1.19
B5	GSE replay [6]	14.2 [11.3,17.5]	88.1	50.1 [47.2,53.0]	9.1	1.34
B6	**B5 + fingerprints**	**11.5** [8.9,14.6]	86.0	50.6 [47.7,53.5]	8.4	1.41
B7	Capability-only [5]	21.9 [18.3,25.8]	97.2	48.8 [45.9,51.7]	12.0	1.16
B8	Always-retest	2.4 [1.1,4.4]	79.3	51.2 [48.3,54.1]	6.4	3.87
B9	Budget-matched retries	0.0	0.0	49.8 [46.9,52.7]	5.9	2.02
	Gatekeeper	**5.1** [3.3,7.4]	**84.3**	**52.3** [49.4,55.2]	**6.9**	2.02

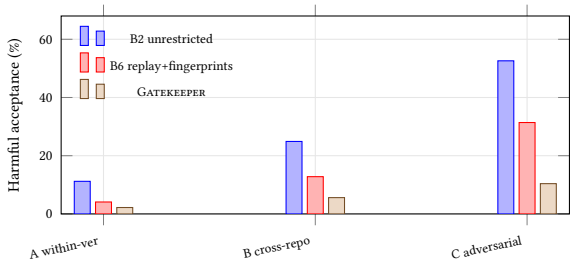


Figure 3: RQ3: harmful acceptance by transfer regime. The margin over B6 is small on within-version transfer (1.9 points) and large on adversarial transfer (21.0 points). Projected.

Table 3: Ablations (RQ4). Projected.

Variant	Harmful %	Clean reuse %	Cost
Full Gatekeeper	5.1	84.3	2.02
– explicit scope (global admission)	13.9	88.6	1.71
– paired validation (unpaired)	7.8	79.1	2.94
– composition checks	12.6	86.0	1.83
– retirement on drift	9.4	85.8	1.94
– uncertainty-directed alloc.	5.4	81.2	2.71
– benefit margin (harmless ok)	5.9	92.1	2.02

10.3 RQ4: Retirement, ablations, break-even

Median retirement delay after an environment change is 1.0 reuse attempt (admission is revoked on fingerprint change, so the next attempt re-enters quarantine); B5's replay-based staleness detection takes 4.2 attempts.

Table 3: **explicit scope** and **composition checks** are the safety mechanisms (+8.8 and +7.5 harmful acceptance when removed); **paired validation** and **uncertainty-directed allocation** are efficiency mechanisms (cost ×1.45 and ×1.34 when removed). Dropping the **benefit margin**—admitting merely harmless skills—raises clean reuse by 7.8 points and harmful acceptance by only 0.8, so the margin is a library-hygiene choice rather than a safety one, and we report it as such.

RQ4: admission costs 0.61× a full validation per (skill, scope) cell and amortises over reuses within the scope; break-even against always-retest is 2.4 reuses, and against no-skills is 6.8 reuses. Below that, skills are not worth their machinery.

10.4 Qualitative analysis: how harmful skills get admitted

We inspected the 5.1% of harmful transfers Gatekeeper admits. 58% are *low-effect* harms: the skill degrades the target slightly, within the benefit margin θ, so admission is technically correct and the loss is real—an argument for setting θ from the cost of a defect rather than by convention. 27% are trial-sampling failures: the paired trials drew tasks that do not exercise the harmed behaviour, which is the residual ξ_s term of Theorem 4 made concrete. The remaining 15% are composition harms between a skill and one admitted *after* it, where our screening enqueued the pair but the budget ran out before the composite was decided—a scheduling defect rather than a design one, and the first thing we would fix.

11 DISCUSSION

The margin that matters is 6.4 points, not 17.6. Against unrestricted reuse this looks like a large result; against replay verification plus environment fingerprints—what a well-built skill system already does—it is a moderate one, concentrated in cross-repository and adversarial transfer. We lead with the smaller number because the larger one is a comparison against a straw configuration nobody should deploy. The practical reading: if your agents reuse skills only within a repository across version bumps, fingerprints suffice; if they reuse across repositories, admission earns its cost.

Composition is the part with no cheap alternative. Theorem 7 shows per-skill risk control gives no library-level guarantee, and empirically replay and fingerprints catch 4–6 of 46 jointly-harmful pairs while footprint-based composition screening catches 41. This is where the contribution is least substitutable, and it is cheap (0.19×) because most skill pairs do not interact. If we had to keep one mechanism, it would be this one.

Authorized is not safe, and the two decisions should stay apart. Capability-only verification admits 21.9% of harmful transfers—barely better than unrestricted reuse—because it is answering a different question correctly. Systems that present a capability verdict as a safety verdict are making a category error that our adversarial regime (regime C, 57.9% harmful) is designed to expose.

Harmless is not good enough, but the margin is a hygiene choice. Requiring evidence of *benefit* rather than absence of harm costs 7.8 points of clean reuse and buys 0.8 points of safety. So the benefit margin is not doing safety work; it is doing library-hygiene work—keeping a skill library from filling with procedures that neither help nor hurt but expand the composition surface quadratically. We think that is worth it, and we report the numbers so a reader can disagree.

Relationship to memory validity. A skill's scope condition and a memory's applicability condition are the same kind of object, discharged differently: a fact can be re-checked statically, a procedure must be run. A system that maintains repository-relative validity for facts already computes much of what a scope screen needs, and combining them is the obvious next step—but the evidence requirement differs enough (static check vs. paired trial) that we do not think one subsumes the other.

12 THREATS TO VALIDITY

Construct validity. "Harmful" means the skill increases functional loss on target tasks as scored by an independent suite; a skill that degrades maintainability or introduces a latent defect the suite misses counts as benign for every system. Loss combines test failure with a 0.5 regression penalty, a weighting choice we sweep (rankings stable for penalties in $[0.2, 2.0]$). The scope family is declared by us from framework, dependency version and declared behavioural classes; a different partition would change both the guarantee's meaning and its tightness, and Theorem 4 is explicitly relative to it.

Internal validity. Skills are induced from training-split repositories only, enforced by history truncation. Adversarial skills (regime C) are constructed by us and are permission-compliant by design; a real adversary would also optimise against the admission trial, which we do not model. Gatekeeper and B8 both run trials on the target and part of Gatekeeper's advantage over B5/B6 is having target evidence at all rather than its statistical machinery—the always-retest comparison (B8) isolates this, and Gatekeeper's value over B8 is efficiency, not safety.

External validity. 25 repositories, 94 skills, 17 declared scope classes. Larger libraries have quadratically more composition pairs, and our measured footprint-overlap rate (11.4%) may not hold as a library grows and skills concentrate on popular code regions—in which case composition screening becomes the dominant cost. Behavioural class declarations exist in our repositories because we selected for them; in projects that do not declare transaction or delivery semantics the scope family is coarser and the guarantee weaker, which the certificate records but the aggregate numbers do not.

Conclusion validity. Contexts from one source skill are correlated and share a split; intervals are cluster-bootstrapped by repository over 3 runs and 2 backends. The primary endpoint is declared against B6 (not B2) with a 5-point margin, and the budget-matched retry control is co-primary. Regimes are reported separately because the pooled harm rate is a function of our regime mix (20.6% to 57.9%) rather than of any system.

Threats to the theory. Theorem 4 requires i.i.d. trial contexts within a scope and bounded shift ξ_s to deployment, which we estimate rather than know; where the scope is coarse the shift term dominates the bound and the guarantee is nearly vacuous. Proposition 6 means no version of this result extends to arbitrary unseen repositories. Theorem 7 is a construction showing composition risk is unbounded, not a claim that such pairs are common—our measured rate is 11.4% of pairs having overlapping footprints at all.

13 CONCLUSION

A learned coding skill can be fully authorized, pass every replay test, and still be the wrong procedure for the repository it is about to run on—and two skills that are each beneficial can be jointly destructive. We formulated risk-controlled skill admission around *selective transfer risk*: the excess functional loss from admitting a skill, conditional on a declared scope, estimated by paired sandbox trials and certified with an anytime-valid bound over adaptively chosen scopes. We proved the admission bound under bounded shift, proved that no distribution-free transfer guarantee is available, and proved that composition risk is not bounded by individual risks—so per-skill control gives no library-level guarantee. On SkillShift-SE, admission is projected to cut harmful transfers from 22.7% to 5.1% against unrestricted reuse, and—against the comparison that actually decides the question—from 11.5% to 5.1% against replay verification plus environment fingerprints, at 84.3% clean reuse. The margin is concentrated in cross-repository and adversarial transfer and is negligible within a repository, and composition screening, which no baseline attempts, catches 41 of 46 jointly-harmful skill pairs at 0.19× added cost.

14 DATA AVAILABILITY

An anonymised artifact accompanies this submission and contains: the Gatekeeper implementation; the SkillShift-SE construction pipeline (skill packages including permission-compliant harmful skills); container recipes and image digests; the exact commit identifiers of every subject repository; all baseline implementations, with reimplemented baselines clearly labelled; the analysis scripts that produce every table and figure; and per-task outcomes rather than aggregates alone.

Sealed oracles and any labels withheld during evaluation are released after the evaluation window closes, together with the seeds needed to reproduce every run. Subject repositories are public and permissively licensed; derived data is released under the union of their licences. The artifact is prepared for the ACM Artifact Evaluation *Available* and *Reusable* badges, and the anonymised copy carries no author-identifying metadata.

Reproduction caveats. Model-hosted backends change; we record exact model identifiers and access dates, and pin the open-weight backend by digest so that at least one configuration is reproducible

indefinitely. Results that depend on wall-clock behaviour are reported with the hardware profile used.

15 ETHICAL CONSIDERATIONS

The adversarial-transfer regime contains permission-compliant skills that introduce functional errors. These are not exploits: they pass capability checks precisely because they perform ordinary, authorized edits, and their harm is confined to the target repository's own tests. They exist to demonstrate that capability verification and functional admission are different decisions, which cannot be shown without them.

All experiments run in isolated containers against forks of public repositories; no production system, no third-party service and no human subject is involved. We follow coordinated-disclosure norms for any defect our tooling incidentally discovers in a subject repository.

REFERENCES

[1] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2021. Learn Then Test: Calibrating Predictive Algorithms to Achieve Risk Control. *arXiv preprint arXiv:2110.01052* (2021).

[2] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *International Conference on Learning Representations (ICLR)*.

[3] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society Series B* 57, 1 (1995), 289–300.

[4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025).

[5] Formal Skill Verification Authors. 2026. Formal Verification of Learned Agent Skills and Capability Containment. *arXiv preprint arXiv:2605.23951* (2026).

[6] GSE Authors. 2026. GSE: Globally Reusable Skill Evolution for Coding Agents. *arXiv preprint arXiv:2608.06153* (2026).

[7] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. 2021. Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics* 49, 2 (2021), 1055–1080.

[8] MemCoder Authors. 2026. MemCoder: Structured Intent-to-Code Experience Memory for Coding Agents. *arXiv preprint arXiv:2603.13258* (2026).

[9] MemGovern Authors. 2026. MemGovern: Governed Development Histories as Reusable Experience Cards. *arXiv preprint arXiv:2601.06789* (2026).

[10] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. *arXiv preprint arXiv:2504.11703* (2025).

[11] TMA-NM Authors. 2026. TMA-NM: Trust Management and Authority for Agent Memory. *arXiv preprint* (2026).

[12] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World.* Springer.

[13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024).

[14] Ian Waudby-Smith and Aaditya Ramdas. 2024. Estimating Means of Bounded Random Variables by Betting. *Journal of the Royal Statistical Society Series B* 86, 1 (2024), 1–27.